

Computing the Dot Product of Two Vectors in C and Java

Introduction

This project implements and compares two programs for computing the dot product of two vectors, one in C and one in Java. The goal is to study how programming language, data type, and vector length affect performance. The tested data types are `int`, `short`, `signed char`, `float`, and `double`. For Java, `byte` is used as the closest counterpart of C `signed char`, since Java does not provide a `signed char` type.

The main purpose of this experiment is not only to compare C and Java in general, but also to observe how performance changes across different numeric types and input sizes. In addition, the project investigates how overflow-related test settings and execution order can affect benchmarking results.

Method

Two separate programs were implemented: `dotproduct.c` and `Dotproduct.java`. Each program generates two random vectors, computes their dot product, performs warm-up runs, and then records average execution time over repeated measurements.

For each tested vector length, the following steps are performed:

1. Generate two arrays of the same type and length.
2. Execute several warm-up runs.
3. Execute several measured runs.
4. Compute the average runtime in nanoseconds.
5. Print the result both as a formatted table in the terminal and as a CSV file.

Timing Methods

- In C, `clock_gettime(CLOCK_MONOTONIC, ...)` is used.
- In Java, `System.nanoTime()` is used.

These timing methods provide finer resolution than crude timing functions and are suitable for microbenchmark-style measurements.

In the C source file, the line

```
#define _POSIX_C_SOURCE 200809L
```

is used before including system headers. This enables the POSIX.1-2008 interface level so that functions such as `clock_gettime()` and constants such as `CLOCK_MONOTONIC` are properly exposed by the system headers. In this project, that definition is necessary for using a high-resolution monotonic timer in a portable POSIX-style way.

Warm-up and Measurement

Warm-up and measurement execute the same dot-product computation. The difference is that warm-up runs are used to bring the program into a more stable state before formal timing. This is especially important in Java because the JVM may still be performing JIT-related optimization during early executions.

Experimental Environment

The experiments were conducted on the same machine for both C and Java so that the hardware conditions remained consistent across the comparison. The C program was compiled with GCC using the `-O3` optimization flag, and the Java program was compiled and executed with the standard JDK toolchain.

The main hardware information is:

- CPU: 13th Gen Intel(R) Core(TM) i9-13980HX @ 2.20 GHz
- RAM: 16.0 GB total, 15.6 GB available

The following environment information should be recorded together with the final submitted results:

- CPU model
- Memory size
- Operating system
- GCC version
- Java version
- Compilation commands

For this project, the main compilation settings are:

```
gcc -O3 -std=c11 dotproduct.c -o dotproduct -lm
javac Dotproduct.java
```

Using the same machine and fixed compilation settings helps reduce unnecessary variation and makes the comparison more reproducible.

Experimental Design

Vector Lengths

The tested vector lengths are:

- 1,000
- 10,000
- 100,000
- 1,000,000
- 5,000,000
- 10,000,000

These sizes were chosen so that both small-scale and large-scale behavior could be observed.

Safe and Overflow Groups

For integer types, two input groups were used:

- `safe`
- `overflow`

The `safe` group was designed to reduce the chance of overflow by controlling the value range according to the vector length. The lower bound was derived from:

$$\sqrt{\text{type_max} / n}$$

This means that when the vector length becomes larger, the safe input magnitude becomes smaller.

The `overflow` group was introduced as a comparison group. The motivation was that if overflow were avoided by widening operands or accumulators to larger types, then the arithmetic would no longer strictly remain in the original tested type. That would make it harder to discuss the speed of each original type itself. Therefore, the overflow group was kept as a native same-type arithmetic case for performance observation, even though the numerical result may wrap around or become unreliable.

For the overflow group, each element magnitude was chosen from:

$$[\sqrt{\text{type_max} / n}, \text{type_max}]$$

and then assigned a random sign. This design ensures that overflow-group values stay in a genuinely large-value range, instead of frequently falling back to small magnitudes.

Floating-point Range

For `float` and `double`, the final safe input range was fixed to:

$$[-1.0, 1.0]$$

Earlier attempts to derive floating-point bounds from the maximum representable value produced unrealistically large numbers and made the results difficult to interpret. Therefore, a small and stable range was used instead.

Only the `safe` group was kept for floating-point types. The earlier overflow-style design for floating-point values was removed because extremely large values made the output harder to interpret and did not provide a useful comparison for the main goal of the experiment.

Randomized Execution Order

At an earlier stage, the safe group was always executed before the overflow group. In that version, the overflow group often appeared slightly faster. This raised the concern that the difference might be caused by execution order rather than by the data range itself.

To reduce this bias, the integer `safe` and `overflow` runs were later executed in random order during both warm-up and measurement. If one group had already reached its required run count, the remaining runs were assigned to the other group. After this change, the consistent advantage of the overflow group became much less obvious. This suggests that the earlier pattern was largely caused by run-order effects such as cache state, warm-up progress, or JVM behavior.

Language-Specific Note

The C implementation uses `signed char`, while the Java implementation uses `byte` as the closest equivalent. This is a reasonable approximation because both are 8-bit signed integer types. However, they are not exactly identical in language semantics. In Java, arithmetic on `byte` is first promoted to `int`, and then cast back when needed. This may partly explain why the Java `byte` case does not necessarily perform better than `int`.

Results and Analysis

Result Presentation

The full benchmark output was generated both as formatted terminal tables and as CSV files. In the final PDF report, it would be helpful to include at least one table or figure so that the trends can be observed more directly.

Useful presentation choices include:

- A summary table comparing C and Java for each data type at selected vector lengths.
- A line chart of runtime versus vector length for `int`.
- A line chart comparing `short` and `signed char` between the two languages.
- A line chart comparing `float` and `double` between the two languages.

Even if only part of the data is visualized, adding at least one table or figure would make the discussion clearer than using text alone.

Table 1 lists a representative subset of the measured average runtimes for the largest vector length (`n = 10,000,000`), which is useful because large inputs make the overall trend easier to observe.

Type	Mode	C measure avg (ns)	Java measure avg (ns)
int	safe	3548990	2843446
int	overflow	3794345	2822315
short	safe	1406917	5215044
short	overflow	1643406	5370386
signed char	safe	867160	4772308
signed char	overflow	665916	4630998
float	safe	4573856	3992549
double	safe	6920419	6847916

This table shows the same overall pattern as the full output files: C remains clearly faster for narrower integer types, while Java gets much closer for `int`, `float`, and `double` at large vector sizes.

The benchmark results show several general trends.

First, C is usually faster for integer-like types, especially `short` and `signed char`. This is consistent with the expectation that native compiled code often has lower runtime overhead.

Second, Java becomes more competitive for `int` , `float` , and `double` , especially for larger vector sizes. After warm-up, the JVM can optimize frequently executed code, which reduces the gap between Java and C.

Third, as vector length increases, the runtime generally increases in an approximately linear way. For very large vectors, memory access cost and memory bandwidth become more important. At that point, language-level differences may become smaller than at small input sizes.

Fourth, in Java, narrower integer types such as `byte` and `short` do not necessarily outperform `int` . One reason is that Java promotes these narrower types to `int` during arithmetic, and then truncates them back when necessary. This means that the smaller source type does not automatically imply a cheaper execution path.

Fifth, warm-up is especially important in Java. In many cases, warm-up averages are higher than measured averages, which suggests that the JVM is still stabilizing or optimizing during early runs.

Warm-up Observation

The results provide a useful observation about warm-up behavior. In C, warm-up and measured times are often relatively close, although some differences still appear because of cache state, page faults, or normal runtime noise. In Java, however, warm-up effects are more noticeable. In many cases, the warm-up average is higher than the measured average, which is consistent with JVM startup behavior and JIT optimization.

This confirms that including warm-up runs was necessary, especially for Java. Without warm-up, the measured runtime would be more strongly affected by cold-start effects and would be less suitable for cross-language comparison.

Interpretation of the Overflow Group

The overflow group should not be interpreted as a correctness-oriented group. It is mainly a performance-oriented comparison setting. In particular:

- For C, signed integer overflow is undefined behavior according to the language standard.
- For Java, signed integer overflow is defined as two's-complement wraparound.

Therefore, the overflow-group results are mainly useful for observing performance behavior under native same-type arithmetic, not for validating numerical accuracy.

Limitations

This experiment provides useful comparative results, but several limitations should be noted.

First, the benchmark is still sensitive to runtime noise. Background processes, CPU frequency changes, cache state, and operating system scheduling may all affect the measured time, especially for smaller vector sizes.

Second, the Java runtime introduces effects such as JIT compilation and garbage collection, which can make timing less stable than in a pure native execution environment.

Third, the overflow group is not a correctness-oriented configuration. In particular, signed integer overflow in C is undefined behavior according to the language standard, so those results should be interpreted carefully.

Fourth, only a limited number of warm-up and measured runs were used. More repetitions, or repeated full benchmark rounds followed by median selection, could further improve stability.

Conclusion

This project implemented dot-product benchmarks in both C and Java and compared their performance across multiple data types and vector lengths.

The overall results show that C tends to have an advantage for integer-based computations, especially narrower integer types. Java, however, becomes more competitive after warm-up and can approach C performance for `int` , `float` , and `double` at large input sizes.

The experiment also showed that benchmark design details matter. Overflow handling, input range selection, and execution order can all influence measured performance. In particular, randomizing the execution order of safe and overflow cases improved the fairness of the comparison and reduced misleading order-related effects.

In summary, the project demonstrates not only differences between C and Java, but also the importance of careful benchmarking methodology when interpreting performance results.